Slam-Toolbox Mapping

1. Course Content

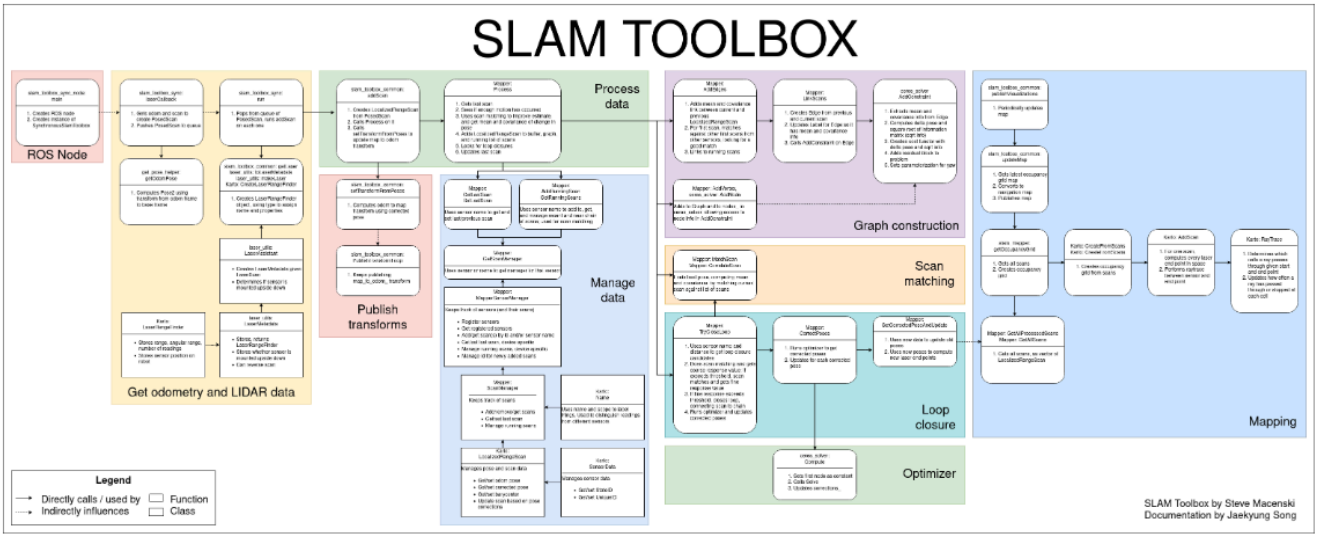
- Master using SLAM-Toolbox (online synchronous mode) for 2D laser SLAM mapping.
- Master remote controlling the A1 Pro robot for mapping and viewing laser scan matching and pose graph optimization process in RViz2.
- Master methods for saving standard OccupancyGrid raster maps (`.yaml` + `.pgm`) and SLAM-Toolbox pose graph files (`.posegraph` + `.data`).

---

2. SLAM-Toolbox Introduction

SLAM-Toolbox is an open-source 2D SLAM (Simultaneous Localization and Mapping) algorithm package officially recommended and deeply integrated by ROS 2, led by ROS navigation and localization working group expert Steve Macenski. It is based on **Graph-based SLAM** theoretical framework, designed for mapping and localization in long-cycle, large-scale dynamic environments.

- **Front-End Scan Matching**: Through continuous scan buffer and correlative scan matching (CSM / Karto Scan Matcher), multiple frames of LaserScan lidar point cloud are quickly registered with odometry prior to generate continuous pose nodes.
- **Back-End Graph Optimization**: Background constructs pose graph based on Google Ceres nonlinear optimization solver, and when loop closure is triggered, combines HuberLoss robust kernel function to perform global pose graph relaxation and optimization, eliminating accumulated drift.
- **Features and Advantages**:
    - **Lifelong/Continuing Mapping**: Supports loading existing pose graph files and continuing to expand and correct on historical maps.
    - **Map Serialization and Deserialization**: Natively supports saving complete graph structure (pose nodes and original scan point cloud) as `.posegraph` and `.data` files.
    - **Low CPU Usage and High Real-time Performance**: Compared to traditional algorithms, multi-threaded architecture and sparse graph optimization mechanism greatly reduce computational overhead, very suitable for embedded edge computing platforms.



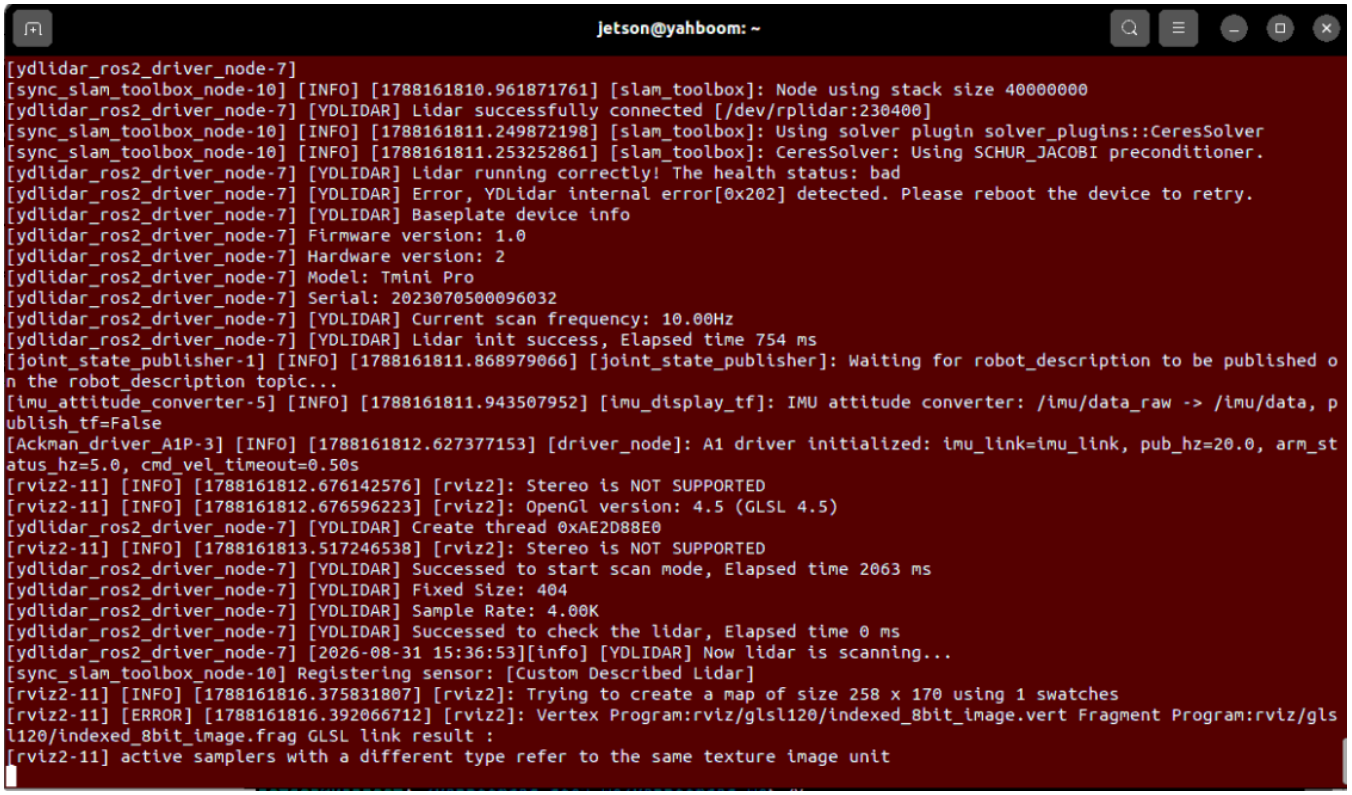
3. Preparation and Running

This example requires opening 3~4 terminals to start the following nodes in sequence:

3.1 Start SLAM-Toolbox Mapping Node

Run in **Terminal 1** (automatically starts chassis driver, lidar driver, and SLAM-Toolbox core algorithm):

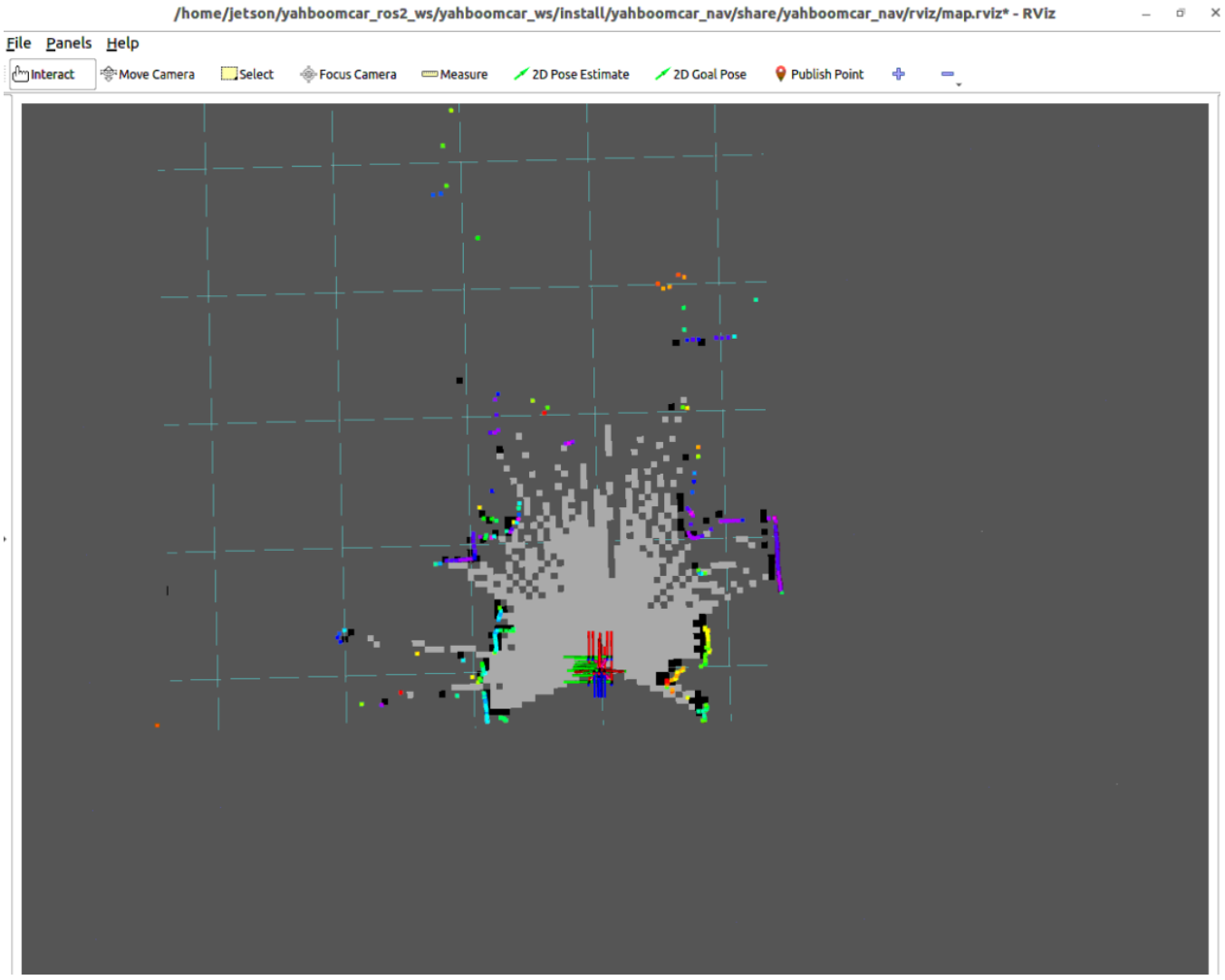
```
ros2 launch yahboomcar_nav map_slam_toolbox_launch.py
```



3.2 Start RViz Map Visualization

Run in **Terminal 2** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

```
ros2 launch yahboomcar_nav display_map_launch.py
```



3.3 Start Keyboard Control to Move Robot for Mapping

Run in **Terminal 3** the keyboard control node:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 run yahboomcar_ctrl yahboom_keyboard

Control Your SLAM-Bot & Robotic Arm!
-----------------------------------
Moving around:
   u    i    o
   j    k    l
   m    ,    .

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%
t/T : x and y speed switch
s/S : stop keyboard control
space key, k : force stop

Arm Control (publish to 'arm_joint'):
1 - 6 : Select Joint 1~6
a / d : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
+ / - : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
0     : Reset all joints to initial pose

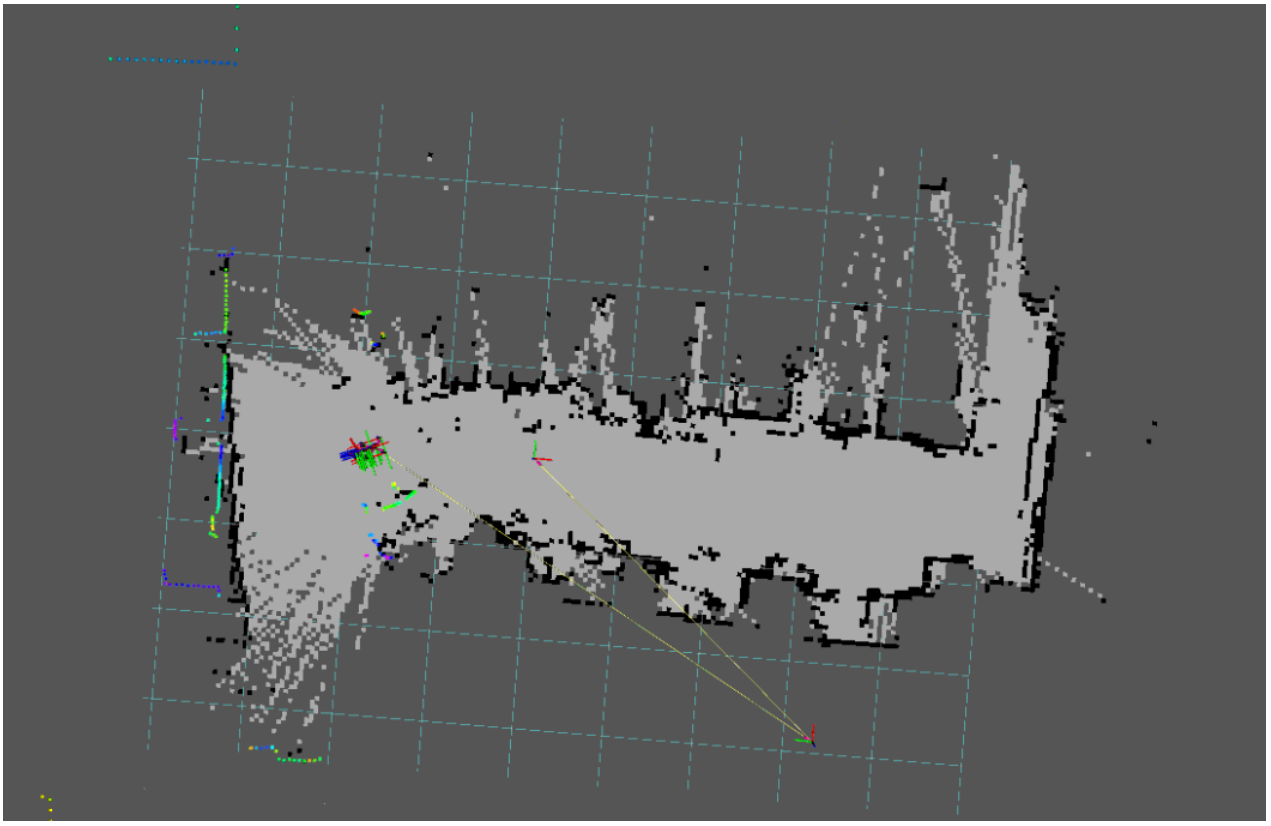
Angle Limits:
Joint 1~4 : 0° ~ 180°
Joint 5   : 0° ~ 270°
Joint 6   : 0° ~ 180°

anything else : stop smoothly
CTRL-C to quit

currently:      speed 0.2       turn 1.0
Current Joint Selection: Joint 1 (90°)
currently:      speed 0.18000000000000002       turn 0.9
currently:      speed 0.16200000000000003       turn 0.81
currently:      speed 0.14580000000000004       turn 0.7290000000000001
```

3.4 Mapping Operation and Map Save Instructions

1. **Mapping Driving Suggestions**:
    - Remote control the robot to slowly and smoothly drive through the entire environment to be tested, observe the continuous matching of laser point cloud and real-time expansion of occupancy grid map in RViz.
    - Appropriately reduce speed at turns to allow the lidar sufficient frame rate to update the local buffer.
    - After driving one loop and returning near the starting point to form a loop closure, the system automatically triggers global loop closure detection and optimizes the entire pose graph, eliminating corridor accumulated error.



2. **Save 2D Raster Map (.yaml + .pgm)**:
   After mapping is complete, run the raster map save command in **Terminal 4**:

```
ros2 launch yahboomcar_nav save_map_launch.py map_name:=map
```

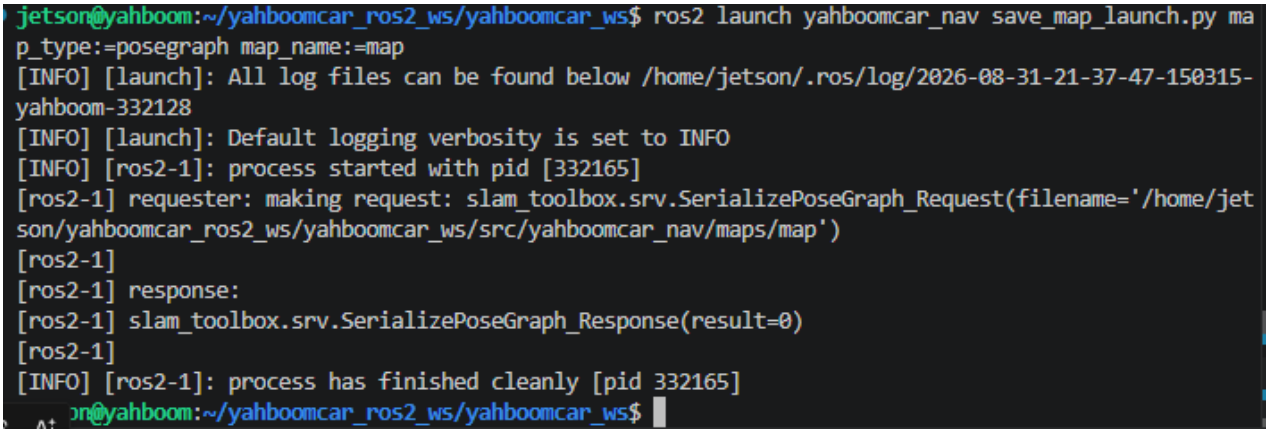
```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 launch yahboomcar_nav save_map_launch.py ma
p_name:=map
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-08-31-21-35-47-350487-
yahboom-326961
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [map_saver_cli-1]: process started with pid [327023]
[map_saver_cli-1] [INFO] [1788183347.680329957] [map_saver]:
[map_saver_cli-1]       map_saver lifecycle node launched.
[map_saver_cli-1]       Waiting on external lifecycle transitions to activate
[map_saver_cli-1]       See https://design.ros2.org/articles/node_lifecycle.html for more informati
on.
[map_saver_cli-1] [INFO] [1788183347.681596944] [map_saver]: Creating
[map_saver_cli-1] [INFO] [1788183347.686805537] [map_saver]: Configuring
[map_saver_cli-1] [INFO] [1788183347.692602886] [map_saver]: Saving map from 'map' topic to '/home/
jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map' file
[map_saver_cli-1] [WARN] [1788183349.452905245] [map_io]: Image format unspecified. Setting it to:
pgm
[map_saver_cli-1] [INFO] [1788183349.453067778] [map_io]: Received a 205 X 112 map @ 0.05 m/pix
[map_saver_cli-1] [INFO] [1788183349.471050085] [map_io]: Writing map occupancy data to /home/jetso
n/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pgm
[map_saver_cli-1] [INFO] [1788183349.474274259] [map_io]: Writing map metadata to /home/jetson/yahb
oomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
[map_saver_cli-1] [INFO] [1788183349.474493402] [map_io]: Map saved
[map_saver_cli-1] [INFO] [1788183349.474514555] [map_saver]: Map saved successfully
[map_saver_cli-1] [INFO] [1788183349.477680391] [map_saver]: Destroying
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 327023]
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$
```

Map files will be saved in `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/` directory, including:

- `map.yaml`: Map metadata configuration file (including resolution, origin pose, occupancy

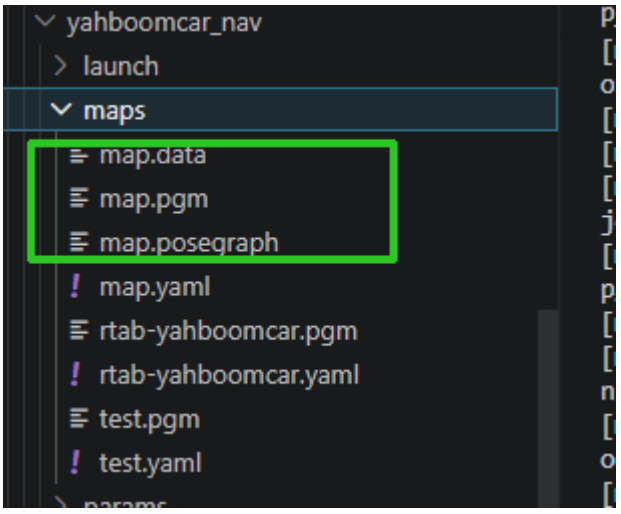
threshold, etc.).
    - `map.pgm`: Grayscale raster image file (white for free traversable areas, black for obstacles, gray for unknown areas).
3. **Save SLAM-Toolbox Pose Graph File (.posegraph + .data)**:
   The pose graph file contains complete topological graph nodes, loop closure constraints, and original laser scan data, which is the core data required for subsequent SLAM-Toolbox continuing mapping or pure localization navigation:

```
ros2 launch yahboomcar_nav save_map_launch.py map_type:=posegraph map_name:=map
```



After successful saving, generated files:

- `map.posegraph`: Pose graph topological structure and optimized node metadata.
- `map.data`: Binary file of original laser point cloud scan measurement data for each node.



> **Note**
>
> After saving is complete, press `Ctrl+C` in each terminal to stop the program.

4. Core Source Code and Algorithm Parameter Analysis

4.1 Mapping Launch File Analysis

Launch File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/map_slam_toolbox_launch.py
```

Launch Core Source

```
def generate_launch_description():
    package_dir = get_package_share_directory("yahboomcar_nav")

    # 1. Declare Launch arguments
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "pose_graph",
            default_value="",
            description="Pose graph basename or absolute path, without .posegraph/.data extension. Empty starts a new map."
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "slam_mode",
            default_value="sync",
            choices=["sync", "async"],
            description="SLAM mode selection. Sync is recommended for repeatable mapping."
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "initial_pose_mode",
            default_value="rviz",
            choices=["rviz", "dock"],
            description="How to initialize an existing pose graph: select pose in RViz or start at dock."
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "odom_source",
            default_value="wheel",
            choices=["wheel", "fused"],
            description="Odometry input: 'wheel' uses independent Ackermann wheel odometry; 'fused' uses RF2O/IMU EKF pipeline."
        )
    )

    pose_graph = LaunchConfiguration("pose_graph")
```

```
    slam_mode = LaunchConfiguration("slam_mode")
    initial_pose_mode = LaunchConfiguration("initial_pose_mode")

    # 2. Chassis and lidar driver start (select independent wheel odometry or EKF fused
odometry based on odom_source)
    wheel_bringup_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(package_dir, "launch", "robot", "laser_bringup_launch.py")
        ),
        condition=LaunchConfigurationEquals("odom_source", "wheel"),
    )

    fused_bringup_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(package_dir, "launch", "robot", "laser_bringup_rf2o_launch.py")
        ),
        condition=LaunchConfigurationEquals("odom_source", "fused"),
    )

    # 3. Introduce SLAM-Toolbox algorithm node and relocalization scheduling
    slam_toolbox_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(package_dir, "launch", "slam", "slam_toolbox_wit_launch.py")
        ),
        launch_arguments={
            "pose_graph": pose_graph,
            "slam_mode": slam_mode,
            "initial_pose_mode": initial_pose_mode,
        }.items(),
    )

    return LaunchDescription([
        *declared_arguments,
        wheel_bringup_launch,
        fused_bringup_launch,
        slam_toolbox_launch,
    ])
```

Launch Design Highlights

- `odom_source` **Odometry Source Selection**:
  Defaults to `wheel` (independent Ackermann wheel odometry). Since the lidar field of view is 235° forward, using independent wheel odometry avoids using the same limited FOV lidar data for both RF2O laser odometry and SLAM-Toolbox, preventing error positive feedback coupling.
- `slam_mode` **Sync/Async Mode**:
  Defaults to `sync` (online synchronous mode), ensuring every lidar frame is strictly processed and pose graph node updated, with higher mapping consistency and boundary sharpness.
- **Continuing Mapping and Relocalization Support**:
  Through `pose_graph` and `initial_pose_mode` parameters, supports loading historical `.posegraph` files, and after giving initial pose in RViz via `slam_pose_initializer` node, smoothly expands historical map.

4.2 SLAM-Toolbox Node Scheduling Analysis

File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/slam_toolbox_wit_launch.py
```

`slam_toolbox_wit_launch.py` as the SLAM core scheduling layer has the following core logic:

1. **Pose Graph File Integrity Check**: Automatically verifies whether the specified historical map `.posegraph` and `.data` files are complete.
2. **Dynamic Node Loading**: Dynamically executes `sync_slam_toolbox_node` or `async_slam_toolbox_node` based on `slam_mode`.
3. **Initial Pose Relocalization Access**: When loading historical map and `initial_pose_mode == 'rviz'`, automatically starts `slam_pose_initializer` node, intercepts lidar point cloud and after RViz gives `/initialpose` initial pose, calls `/slam_toolbox/deserialize_map` service to complete relocalization.

4.3 Map Save Mechanism Analysis

File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/save_map_launch.py
```

```
# 1. Save pose graph (map_type == 'posegraph')
save_posegraph = GroupAction(
    condition=IfCondition(PythonExpression(["'", map_type, "' == 'posegraph'"])),
    actions=[
        ExecuteProcess(
            cmd=[
                'ros2', 'service', 'call', '/slam_toolbox/serialize_map',
                'slam_toolbox/srv/SerializePoseGraph',
                ['{filename: "', PathJoinSubstitution([map_save_dir, map_name]), '"}']
            ],
            output='screen'
        )
    ]
)

# 2. Save 2D raster map (map_type == 'gridmap')
save_map = Node(
    package='nav2_map_server',
    executable='map_saver_cli',
    parameters=[{
        'map_subscribe_transient_local': False,
        'save_map_timeout': 10.0,
    }],
    arguments=[
        '-f', PathJoinSubstitution([map_save_dir, map_name]),
```

```
        '--free', '0.196',
        '--occ', '0.65'
    ],
    condition=IfCondition(PythonExpression(["'", map_type, "' == 'gridmap'"]))
)
```

- **Raster Map Save**: Calls `nav2_map_server`'s `map_saver_cli` node, subscribes to `/map` topic and marks grids with occupancy probability greater than 0.65 as obstacles, less than 0.196 as free, outputting `.yaml` and `.pgm`.
- **Pose Graph Save**: Calls `/slam_toolbox/serialize_map` service, SLAM-Toolbox internally serializes the pose graph and original laser frames in memory to disk files.

4.4 SLAM-Toolbox Algorithm Configuration and Core Parameter Interpretation

Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/mapper_params_online_sync.yaml
```

Core Parameter Description Table

Parameter Name	Current Config Value	Core Function and Physical Meaning
`solver_plugin`	`CeresSolver`	Graph optimization back-end nonlinear least squares solver plugin (based on Google Ceres)
`ceres_loss_function`	`HuberLoss`	Robust kernel function for optimization objective, preventing map distortion from edge incorrect loop closure constraints
`mode`	`mapping`	Working mode: `mapping` / `localization`
`resolution`	`0.05`	Occupancy grid map resolution (0.05 meters/pixel)
`min_laser_range` / `max_laser_range`	`0.1 / 10.0`	Shortest and longest effective laser ranging distance for rasterization mapping (meters)
`minimum_travel_distance`	`0.1`	Robot straight-line displacement threshold (meters), insert new pose graph node when moved beyond this value

Parameter Name	Current Config Value	Core Function and Physical Meaning
`minimum_travel_heading`	`0.01745`	Robot rotation displacement threshold (approximately 1°), insert node even for tiny rotations to ensure dense turn point cloud
`scan_buffer_size`	`30`	Local sliding window scan buffer size, similar to Cartographer's Submap submap mechanism
`scan_buffer_maximum_scan_distance`	`5.0`	Maximum spatial allowed distance between laser frames in scan buffer (meters)
`do_loop_closing`	`true`	Whether to enable global loop closure detection and loop graph optimization
`loop_search_maximum_distance`	`4.0`	Maximum candidate spatial search radius for triggering loop matching (meters)
`loop_match_minimum_chain_size`	`12`	Minimum continuous matching frame chain length required for loop matching, strictly filtering occasional false positive matches
`loop_match_minimum_response_fine`	`0.55`	Loop fine matching score threshold (0~1.0), only establishes loop edge when score exceeds this threshold
`distance_variance_penalty`	`0.25`	Penalty factor for displacement variance when geometric features degrade (such as featureless long corridors), tending to trust wheel odometry
`angle_variance_penalty`	`0.25`	Penalty factor for angular variance when geometric features degrade

YAML Configuration File Complete Source

```yaml
slam_toolbox:
  ros__parameters:

    # 1. Ceres graph optimization back-end solver configuration
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: HuberLoss
```

```yaml
# 2. ROS coordinate frame and base topic parameters
odom_frame: odom
map_frame: map
base_frame: base_footprint
scan_topic: /scan
use_map_saver: true
mode: mapping

debug_logging: false
throttle_scans: 1
transform_publish_period: 0.03
map_update_interval: 5.0
resolution: 0.05
min_laser_range: 0.1
max_laser_range: 10.0
minimum_time_interval: 0.5
transform_timeout: 0.2
tf_buffer_duration: 30.
stack_size_to_use: 40000000
enable_interactive_mode: true

# 3. Front-end scan matching and pose graph node insertion strategy
use_scan_matching: true
use_scan_barycenter: true
minimum_travel_distance: 0.1
minimum_travel_heading: 0.01745 # 1 degree
scan_buffer_size: 30
scan_buffer_maximum_scan_distance: 5.0
link_match_minimum_response_fine: 0.2
link_scan_maximum_distance: 1.5
loop_search_maximum_distance: 4.0
do_loop_closing: true

# 4. Loop closure detection (Loop Closure) and constraint construction thresholds
loop_match_minimum_chain_size: 12
loop_match_maximum_variance_coarse: 1.5
loop_match_minimum_response_coarse: 0.45
loop_match_minimum_response_fine: 0.55

# 5. Correlation matching search space parameters
correlation_search_space_dimension: 0.5
correlation_search_space_resolution: 0.01
correlation_search_space_smear_deviation: 0.1

# 6. Loop search space parameters
loop_search_space_dimension: 6.0
loop_search_space_resolution: 0.05
loop_search_space_smear_deviation: 0.03

# 7. Scan matcher and odometry prior penalty factors
distance_variance_penalty: 0.25
angle_variance_penalty: 0.25
fine_search_angle_offset: 0.00349
```

```
    coarse_search_angle_offset: 0.2618 # 15 degrees
    coarse_angle_resolution: 0.01745 # 1 degree
    minimum_angle_penalty: 0.5
    minimum_distance_penalty: 0.5
    use_response_expansion: false
    min_pass_through: 3
    occupancy_threshold: 0.2
```